AI Security Investigation Platform — Architecture

Agent Loop · Inquiry Layer · Vault · Variant Library · State Machine · Persistence

1. System Overview

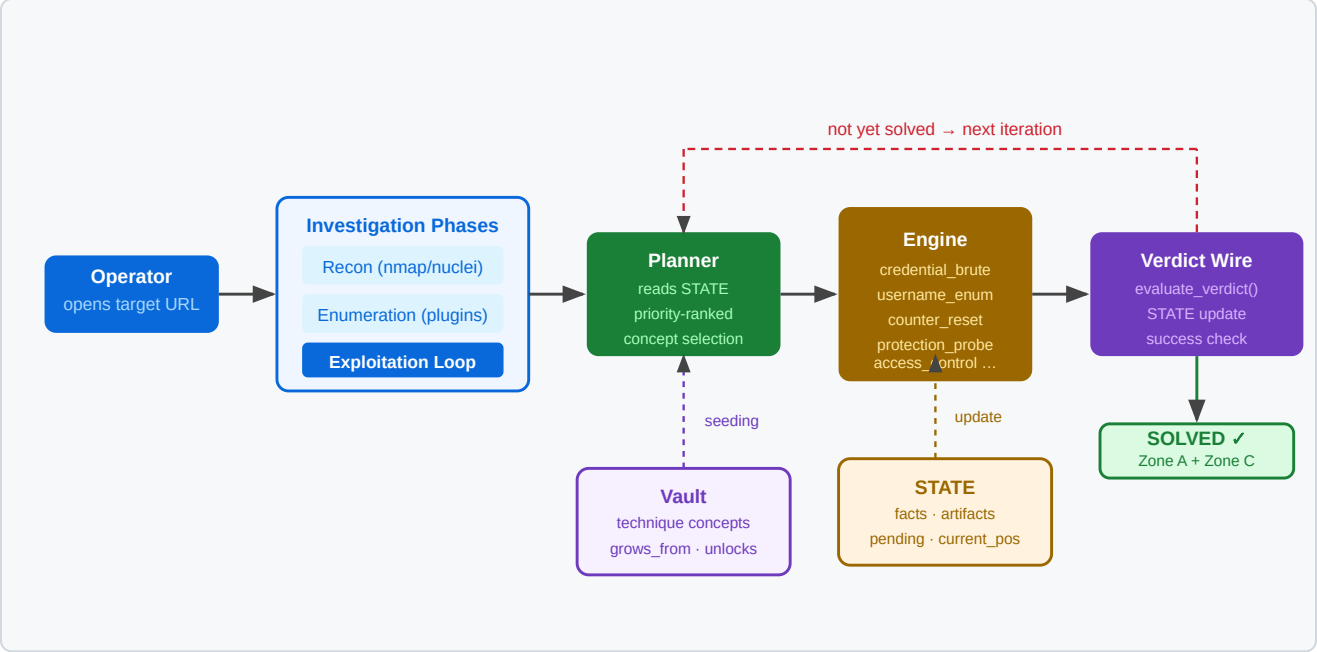
An AI-powered security investigation workbench. The operator opens a target URL; the system runs recon → enumeration → exploitation automatically via a deterministic planner that reads a vault of technique knowledge, selects the appropriate engine variant, and updates shared STATE after each result.

Stack
FastAPI (Python 3.12) · Jinja2 · htmx · Preact + React Flow · SQLite (WAL) · ChromaDB · Ollama Qwen

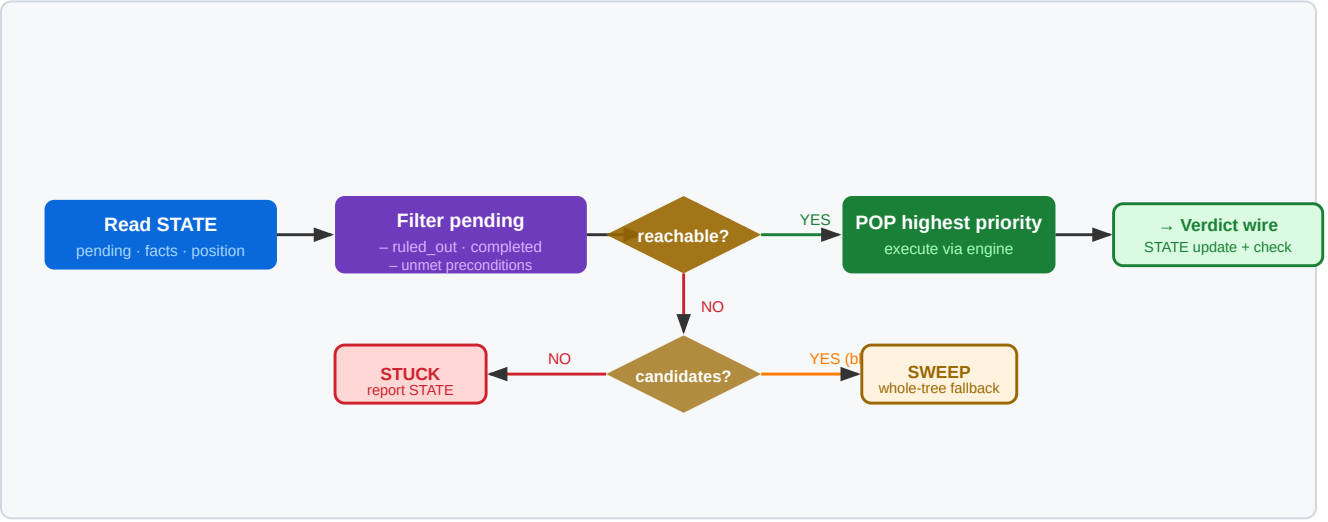
Phases
Recon (nmap/nuclei) → Enumeration (plugins) → Exploitation (agent loop)

Right Panel — 3 Zones
Zone A PROCESS: live STATE (position/facts/artifacts)
Zone B UNKNOWNS: unclassified result count
Zone C AI workspace: chat + planner decisions + SOLVED events

2. End-to-End Process Flow



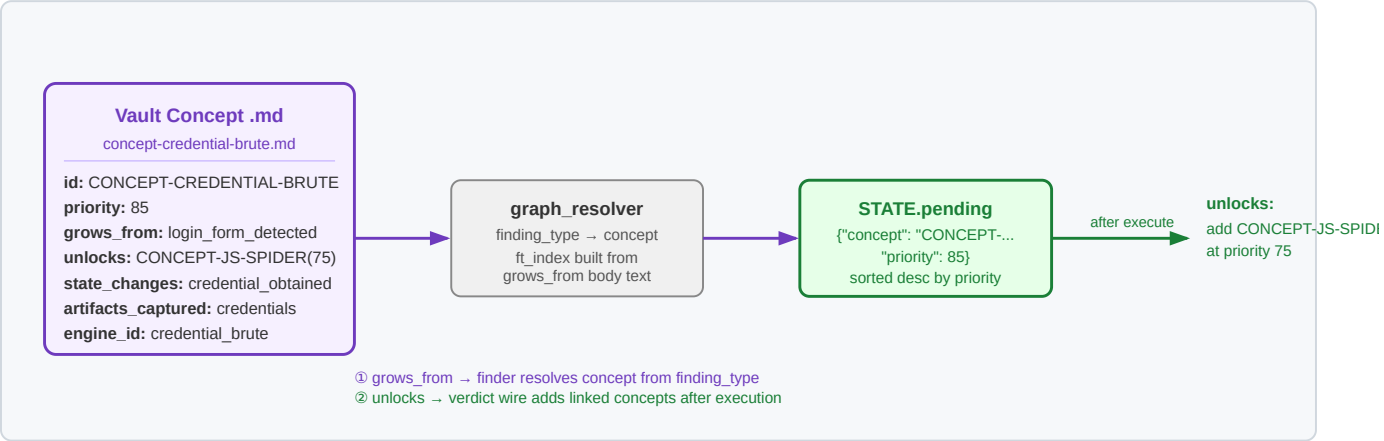
3. Planner Decision Flow



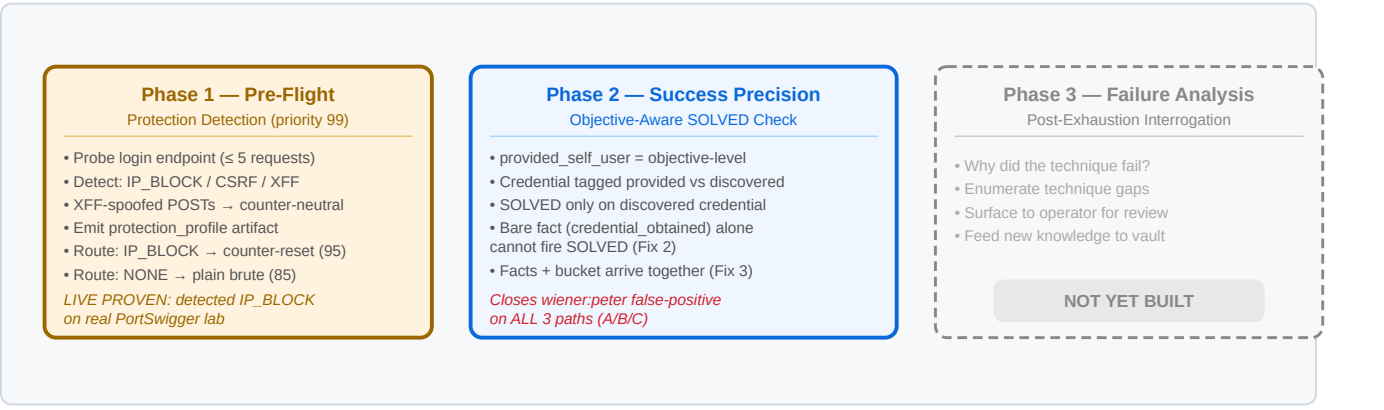
4. STATE Shape (agent_state)

```
// Per-investigation, persisted in validation.db exploitation_branches (state_{inv_id} row)
{
  "goal": {
    "objective": "obtain credentials / access",
    "success_conditions": ["valid_credentials_found"],
    "provided_self_user": null,    // operator's own login — NOT an objective solve
    "target_user": null              // objective account to compromise
  },
  "facts": ["ip_block_detected", "protection_probed", "credential_obtained"],
  "artifacts": {
    "credentials": [{"username": "carlos", "origin": "discovered"}],
    "sessions": [],  "tokens": [],  "files": []
  },
  "pending":   [{"concept": "CONCEPT-CREDENTIAL-BRUTE-COUNTER-RESET", "priority": 95}],
  "ruled_out": [],  "completed": ["CONCEPT-LOGIN-PROTECTION-PROBE"],
  "current_position": {"authentication": "authenticated", "privilege": "none",
                       "access_scope": [], "execution": "none"},
  "planner_log": ["[1] EXECUTE priority=99 [CONCEPT-LOGIN-PROTECTION-PROBE] via login_protection_probe"],
  "solved": true,  "solved_reason": "discovered credentials for 'carlos'"
}
```

5. Vault → Planner Connection



6. Inquiry Layer — Three Phases

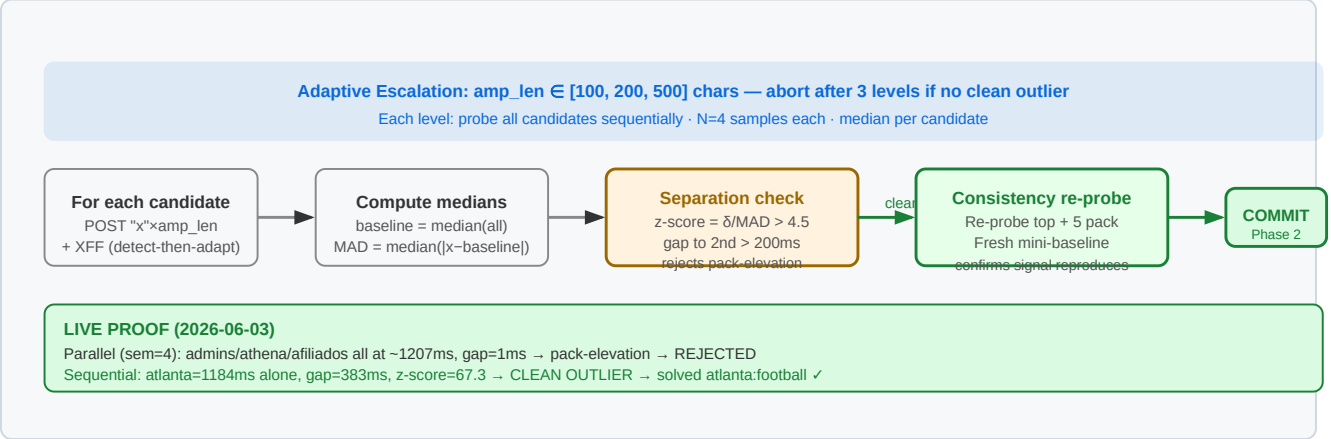


7. Variant Library — Technique Concepts

Concept	Engine	Priority	When used
`CONCEPT-LOGIN-PROTECTION-PROBE`	`login_protection_probe`	99 — first	Always — detects defenses before committing a technique
`CONCEPT-DEFAULT-CREDS`	`default_creds`	90	Quick check — vendor/common credentials
`CONCEPT-CREDENTIAL-BRUTE`	`credential_brute`	85	Plain wordlist brute force on login form
`CONCEPT-USERNAME-ENUMERATION`	`username_enum`	80	Oracle-guided (message/length/status/timing); timing runs sequential
`CONCEPT-CREDENTIAL-BRUTE-COUNTER-RESET`	`credential_brute_counter_reset`	78 → 95*	IP-block with counter-reset bypass; interleaves self-login; *elevated by probe
`CONCEPT-SQLI-AUTH-BYPASS`	`sqli_auth_bypass`	70	SQL injection to bypass authentication
`CONCEPT-ACCESS-CONTROL`	`access_control`	85	IDOR / mass-assignment on discovered API endpoints
`CONCEPT-JS-SPIDER`	`js_spider`	75	Authenticated JS crawl to discover hidden endpoints post-capture
`CONCEPT-NOSQL-OPERATOR-INJECTION`	`auth_bypass_login`	72	MongoDB $ne operator bypass on JSON login

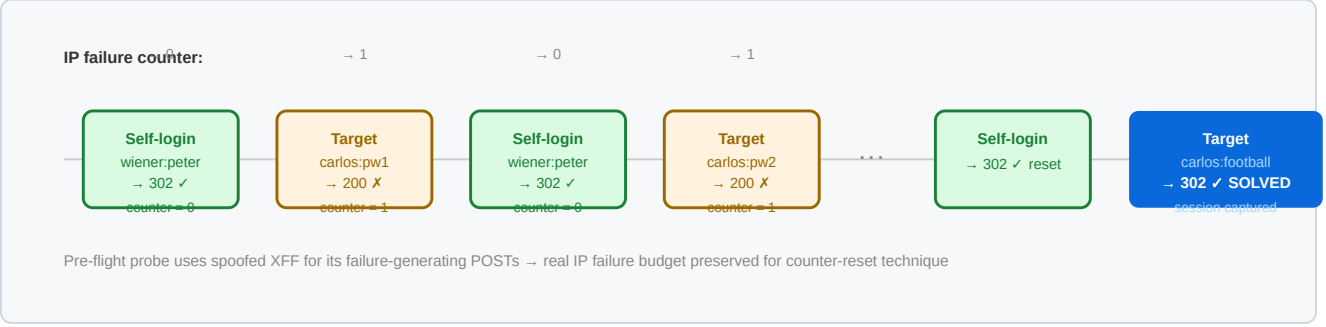
8. Username Enumeration — Timing Oracle (Sequential)

The timing oracle detects a valid username by measuring the bcrypt delay for a long amplified password. Parallel probing corrupts timing via correlated jitter (proven on live lab); the oracle runs **strictly sequential**.



9. Counter-Reset Brute-Force Bypass

Some login forms block after N consecutive failures per IP, but reset the counter on a successful login from the same IP. Bypass: interleave a self-login before every target attempt.



10. Phase 2 — Success Precision Check

FALSE POSITIVE (fixed)
wiener:peter in wordlist → credential_brute captures it → SOLVED fires → WRONG
wiener:peter = operator's own account, NOT the objective target

CORRECT SOLVED
Credential for carlos discovered → origin="discovered" → SOLVED fires →
Zone C: "✓ Solved — discovered credentials for 'carlos'"

Three fixed paths

Path	Gap	Fix
A	engine-scoped provided_self_user	Set at objective level via operator_params
B	fact-fallback bypasses Phase 2	Bare fact alone → False (requires artifact)
C	async: fact without artifact	Credential facts filtered when no artifact captured

check_success logic

```
# _has_discovered_credential(state):
creds = artifacts["credentials"]
if not creds:
    return False  # Fix 2: no bare-fact fallback
provided = goal["provided_self_user"] or ""
for cred in creds:
    if cred.origin == "discovered": return True
    if cred.origin == "provided":  continue
    if not provided: return True  # backward compat
    if cred.username != provided: return True
return False
```

11. Data Persistence

Store	What	Key
`exploitation_branches`	Branch lifecycle per technique run (state, stage_log, artifact)	`branch_id (exb_…)`
`exploitation_branches (state_…)`	Full agent STATE blob per investigation (goal, facts, artifacts, pending, planner_log)	`state_{investigation_id}`
`agent_loop_unknowns`	Unclassified results pending operator review; cross-investigation batch queue	`UUID, indexed by status + investigation_id`
`investigation_chat_history`	Zone C chat messages per investigation; survives page refresh	`UUID, indexed by investigation_id`
`normalized_findings`	Enum findings (login_form_detected, sqli, api_exposure …) — seed planner pending	`finding_id, investigation_id`

Vault concepts	Technique knowledge (grows_from, unlocks, priority, state_changes); operator-editable	knowledge-vault/concepts/*.md
ChromaDB	Semantic embeddings for vault retrieval; BM25 + nomic-embed-text	Container volume /app/knowledge-vault

Cleanup on investigation delete: exploitation_branches · chat_history · agent_loop_unknowns — all deleted in cascade. **Zombie recovery:** on startup, branches stuck at running/armed with no live task → transitioned to failed.

12. Key Invariants & Design Decisions

Invariant	Rationale
Timing oracle: sequential only	Parallel probing creates correlated jitter → pack-elevation masks the real single-user bcrypt signal. Proven on live lab: parallel produced 3-way tie; sequential isolated one user cleanly.
Probe must be counter-neutral	Pre-flight failure-generating POSTs use spoofed XFF so real IP's failure budget is preserved for the counter-reset technique that follows.
SOLVED requires classifiable artifact	Bare credential_obtained fact cannot fire SOLVED. A credential object with a classifiable username must be in artifacts.credentials. Prevents fact-without-artifact false positives.
provided_self_user at objective level	Set via operator_params before any engine runs so every engine's capture is classified correctly — not just the counter-reset path.
No hardcoded lab values	self_user, target_user, self_pass always come from operator_params or STATE goal. No literals in engine code.
Planner pops; engine just runs	The planner selects what to run. The orchestrator (executor) runs it. Engines have no self-direction. Sweep is the fallback when no reachable lead exists.
Facts + artifact arrive together	credential_obtained is filtered from state_changes when no artifact was captured. The two always arrive together or neither does.

13. Known Open Gaps

Counter-reset live-solve (Pieces 2+3)
Both planner-params bug and probe-neutrality bug are fixed. Pending a fresh PortSwigger "Broken brute-force protection, IP block" lab for live end-to-end verification.

self_pass cannot be stored in STATE
Passwords are not persisted. Must be supplied at each planner/run call via operator_params.

Phase 3 — Failure Interrogation
Post-exhaustion analysis (why did the technique fail?) is not yet built. Planned but out of scope for the current build.

Planner param threading
operator_params now thread correctly (Bug 1 fix). Counter-reset can be fully configured via planner/run when operator supplies self_pass.